Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Системы искусственного интеллекта

Лабораторная работа №6

Распознавание рукописных цифр (MNIST)

Выполнил
Чэнь Хаолинь P3316 407960

Преподаватель
Болдырева Елена Александровна

г. Санкт-Петербург, 2026

Содержание

1 Задание **2**

2 Теоретические сведения **2**

3 Подготовка данных **3**

4 Реализация и обучение **4**

4.1 Слои сети 4

4.2 Функция потерь и оптимизатор 5

4.3 Проверка градиентов 5

4.4 Цикл обучения 6

5 Сравнение архитектур **6**

6 Тестовая оценка **9**

7 Распознавание внешних изображений **10**

8 Заключение **10**

1. Задание

Реализовать полносвязную нейронную сеть для распознавания рукописных цифр MNIST. Самостоятельно прочитать данные IDX, реализовать прямой и обратный проход, вычисление функции потерь и обновление параметров методом Adam. Сравнить архитектуры по точности на валидационной выборке, оценить выбранную модель на тесте и проверить её на внешних рисунках.

2. Теоретические сведения

Слой `Flatten` преобразует изображение 28×28 в вектор из 784 элементов. Для батча X слой `Linear` вычисляет $Y = XW + b$, где W — матрица весов, а b — вектор смещений. Для градиента $G = \partial L/\partial Y$ обратный проход имеет вид

$$\frac{\partial L}{\partial X} = GW^\top, \qquad \frac{\partial L}{\partial W} = X^\top G, \qquad \frac{\partial L}{\partial b} = \sum_i G_{i,:}.$$

После скрытого линейного слоя применяется $\mathrm{ReLU}(x) = \max(0, x)$. При обратном проходе она пропускает градиент только для положительных входов. Скрытые веса инициализируются с дисперсией $2/n_{\text{in}}$ по Xe, выходные — с дисперсией $1/n_{\text{in}}$. Смещения равны нулю в начале обучения.

Для логитов z_{ik}, где i — номер примера, k — класс, положим $\tilde{z}_{ik} = z_{ik} - \max_j z_{ij}$. Тогда

$$p_{ik} = \frac{\exp(\tilde{z}_{ik})}{\sum_j \exp(\tilde{z}_{ij})}, \qquad L = \frac{1}{N} \sum_i \left(\log \sum_j \exp(\tilde{z}_{ij}) - \tilde{z}_{i,y_i} \right), \qquad \frac{\partial L}{\partial z_{ik}} = \frac{p_{ik} - \mathbf{1}[k = y_i]}{N}.$$

Вычитание максимума защищает экспоненту от переполнения. `SoftmaxWithLoss` хранит вероятности и one-hot-метки для вычисления градиента.

В режиме обучения Dropout сохраняет каждую активацию с вероятностью $1 - p$ и делит сохранённые значения на $1 - p$, сохраняя их математическое ожидание. В режиме оценки слой пропускает вход без преобразований.

Adam на шаге t обновляет скользящие средние градиента g_t и его квадрата. Для каждого обучаемого параметра θ вычисляются

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$
$$\hat{m}_t = m_t / (1 - \beta_1^t), \qquad \hat{v}_t = v_t / (1 - \beta_2^t), \qquad \theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

3. Подготовка данных

Загрузчик читает заголовки IDX в порядке big-endian, проверяет магические числа 2051 и 2049, размерности и длину массива байтов. Изображения образуют массивы $(60\,000, 28, 28)$ и $(10\,000, 28, 28)$, метки — векторы соответствующих длин. Пиксели преобразуются в `float32` и делятся на 255, метки кодируются в one-hot формы $(N, 10)$.

При seed 42 индексы исходной обучающей выборки перемешиваются. Первые 6 000 объектов используются для валидации, остальные 54 000 — для обучения. Разбиение одинаково для всех моделей. Тестовый набор используется после выбора конфигурации и эпохи.

Таблица 1: Распределение изображений по классам

Выборка	0	1	2	3	4	5	6	7	8	9	Всего
Исходная	5923	6742	5958	6131	5842	5421	5918	6265	5851	5949	60000
Train	5319	6067	5361	5487	5255	4913	5329	5657	5243	5369	54000
Validation	604	675	597	644	587	508	589	608	608	580	6000
Test	980	1135	1032	1010	982	892	958	1028	974	1009	10000

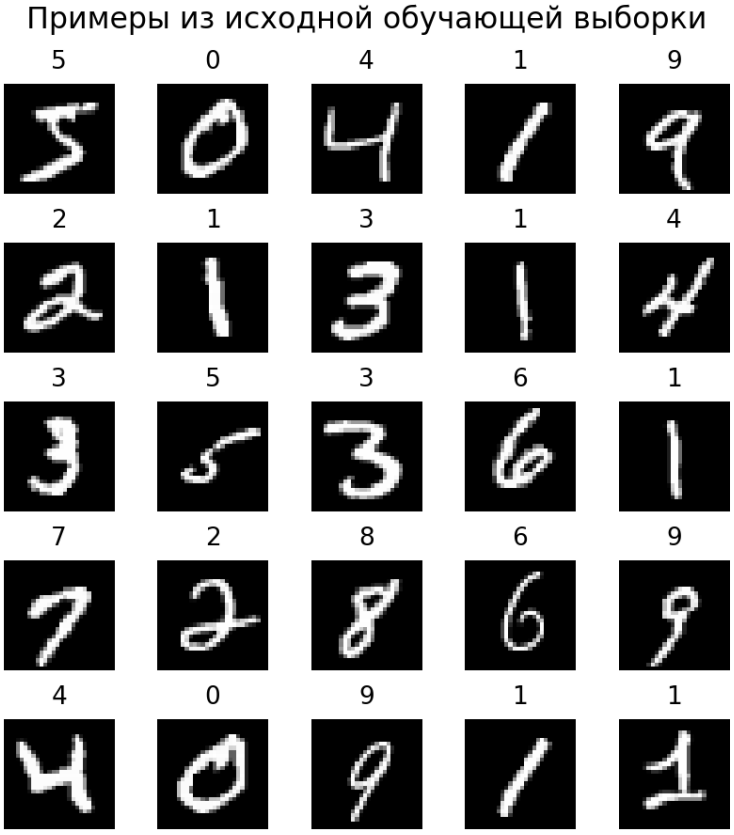


Рис. 1: Первые 25 изображений исходного обучающего набора

4. Реализация и обучение

4.1. Слои сети

Linear хранит вход, ReLU — маску положительных значений, Flatten — исходную форму батча, Dropout — масштабированную маску. Обратный проход использует состояние последнего обучающего прохода.

```
class Linear:
    def __init__(self, n_in, n_out, rng, *, relu=True, dtype=np.float32):
        # He для скрытого слоя с ReLU, масштаб 1/sqrt(n_in) для логитов.
        scale = np.sqrt((2 if relu else 1) / n_in)
        self.W = rng.normal(0, scale, (n_in, n_out)).astype(dtype)
        self.b = np.zeros(n_out, dtype=dtype)
        self.dW = np.zeros_like(self.W)
        self.db = np.zeros_like(self.b)

    def forward(self, x, training=True):
        if training:
            self.x = x
        return x @ self.W + self.b

    def backward(self, dy):
        self.dW[:] = self.x.T @ dy
        self.db[:] = dy.sum(axis=0)
        return dy @ self.W.T

    def parameters(self):
        return ((self.W, self.dW), (self.b, self.db))


class ReLU:
    def forward(self, x, training=True):
        if training:
            self.positive = x > 0
        return np.maximum(x, 0)

    def backward(self, dy):
        return dy * self.positive


class Flatten:
    def forward(self, x, training=True):
        if training:
            self.shape = x.shape
        return x.reshape(len(x), -1)

    def backward(self, dy):
        return dy.reshape(self.shape)


class Dropout:
    def __init__(self, probability, rng):
        if not 0 <= probability < 1:
            raise ValueError("Dropout probability must be in [0, 1).")
        self.keep = 1 - probability
        self.rng = rng

    def forward(self, x, training=True):
        if not training:
            return x
        self.scale = (self.rng.random(x.shape, dtype=np.float32) < self.keep).astype(x.dtype)
        self.scale /= self.keep
        return x * self.scale

    def backward(self, dy):
        return dy * self.scale
```

4.2. Функция потерь и оптимизатор

Кросс-энтропия вычисляется по сдвинутым логитам. Adam обновляет веса и смещения на месте после вычисления градиентов всех слоёв.

```
def _softmax_terms(logits):
    # Вычитание максимума защищает exp от переполнения.
    shifted = logits - logits.max(axis=1, keepdims=True)
    exp = np.exp(shifted)
    total = exp.sum(axis=1, keepdims=True)
    return shifted, exp / total, total


def softmax(logits):
    return _softmax_terms(logits)[1]


class SoftmaxWithLoss:
    def forward(self, logits, one_hot):
        shifted, self.probabilities, total = _softmax_terms(logits)
        self.targets = one_hot
        # log-sum-exp сохраняет конечный loss даже при exp(z) = 0.
        return float((np.log(total[:, 0]) - (one_hot * shifted).sum(axis=1)).mean())

    def backward(self):
        return (self.probabilities - self.targets) / len(self.targets)


class Adam:
    def __init__(self, parameters, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8):
        self.parameters = list(parameters)
        self.lr, self.beta1, self.beta2, self.eps = lr, beta1, beta2, eps
        self.m = [np.zeros_like(value) for value, _ in self.parameters]
        self.v = [np.zeros_like(value) for value, _ in self.parameters]
        self.t = 0

    def step(self):
        self.t += 1
        first_correction = 1 - self.beta1**self.t
        second_correction = 1 - self.beta2**self.t
        for (value, gradient), m, v in zip(self.parameters, self.m, self.v):
            m *= self.beta1
            m += (1 - self.beta1) * gradient
            v *= self.beta2
            v += (1 - self.beta2) * np.square(gradient)
            m_hat = m / first_correction
            v_hat = v / second_correction
            value -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
```

4.3. Проверка градиентов

Для Linear $3 \to 4$ с двумя входными векторами аналитические значения dX, dW и db сравнены с центральными разностями $(L(\theta + h) - L(\theta - h))/(2h)$. При $h = 10^{-5}$ и вычислениях в `float64` максимальная абсолютная ошибка составила $2{,}79 \cdot 10^{-11}$.

4.4. Цикл обучения

Для каждой архитектуры выполнено 12 эпох с мини-батчем 256. Веса и маски Dropout используют генератор с seed 42. Порядок примеров перемешивается в каждой эпохе с использованием генератора с seed 43. Параметры Adam: $\alpha = 0{,}001$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon = 10^{-8}$. Модель выполняет прямой проход по слоям в заданном порядке, а обратный — в обратном. Метрики считаются в конце эпохи на полных train и validation с отключённым Dropout. При равной валидационной точности выбирается более ранняя эпоха.

```
def train(config, train_x, train_y, val_x, val_y, epochs):
    name, widths, dropout = config
    model = MLP(widths, dropout, SEED)
    best_model = MLP(widths, dropout, SEED)
    optimizer = Adam(model.parameters(), lr=LEARNING_RATE)
    objective = SoftmaxWithLoss()
    shuffle = np.random.default_rng(SEED + 1)
    history = []
    best_epoch, best_accuracy = 0, -1.0
    for epoch in range(1, epochs + 1):
        order = shuffle.permutation(len(train_x))
        for start in range(0, len(order), BATCH_SIZE):
            batch = order[start:start + BATCH_SIZE]
            logits = model.forward(train_x[batch], training=True)
            objective.forward(logits, train_y[batch])
            model.backward(objective.backward())
            optimizer.step()

        train_loss, train_acc, _ = evaluate(model, train_x, train_y)
        val_loss, val_acc, _ = evaluate(model, val_x, val_y)
        history.append((name, epoch, train_loss, train_acc, val_loss, val_acc))
        print(f"{name} {epoch:2d}  loss {train_loss:.5f}/{val_loss:.5f}"
              f"  accuracy {train_acc:.4%}/{val_acc:.4%}", flush=True)

        # Strict comparison keeps the earliest checkpoint when accuracy ties.
        if val_acc > best_accuracy:
            best_epoch, best_accuracy = epoch, val_acc
            for (destination, _), (source, _) in zip(best_model.parameters(), model.parameters()):
                destination[...] = source
    return best_model, best_epoch, best_accuracy, history
```

5. Сравнение архитектур

Во всех конфигурациях скрытые Linear сопровождаются ReLU. Переход $A \to B$ меняет ширину первого слоя, $B \to C$ добавляет второй слой, $C \to D$ включает Dropout после каждой ReLU.

Таблица 2: Конфигурации и максимальная точность на валидации

Модель	Скрытые слои	Dropout	Параметры	Эпоха	Точность, %
A	128	0	101 770	12	97,38
B	256	0	203 530	12	97,90
C	256 → 128	0	235 146	12	98,03
D	256 → 128	0,2	235 146	12	98,12

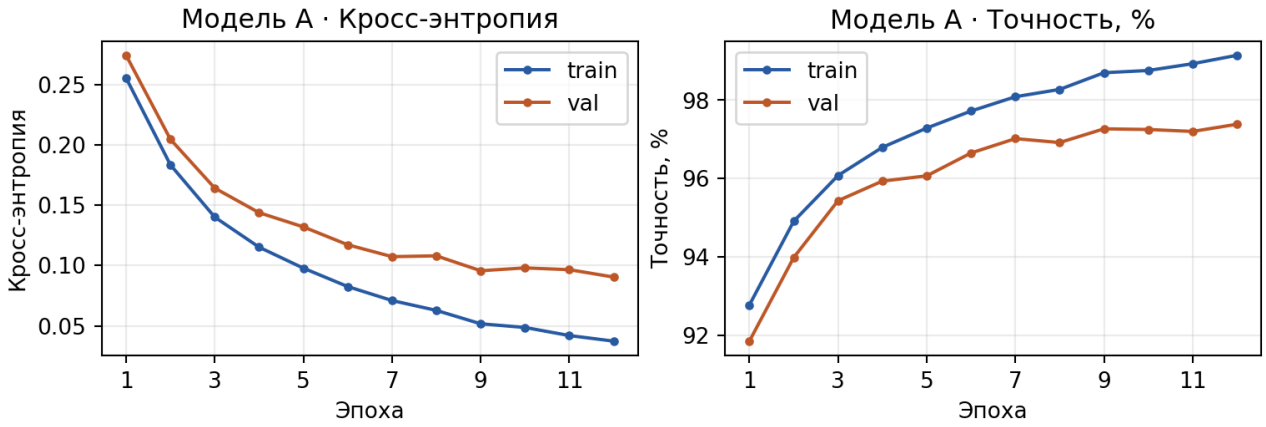


Рис. 2: Потери и точность модели A по эпохам

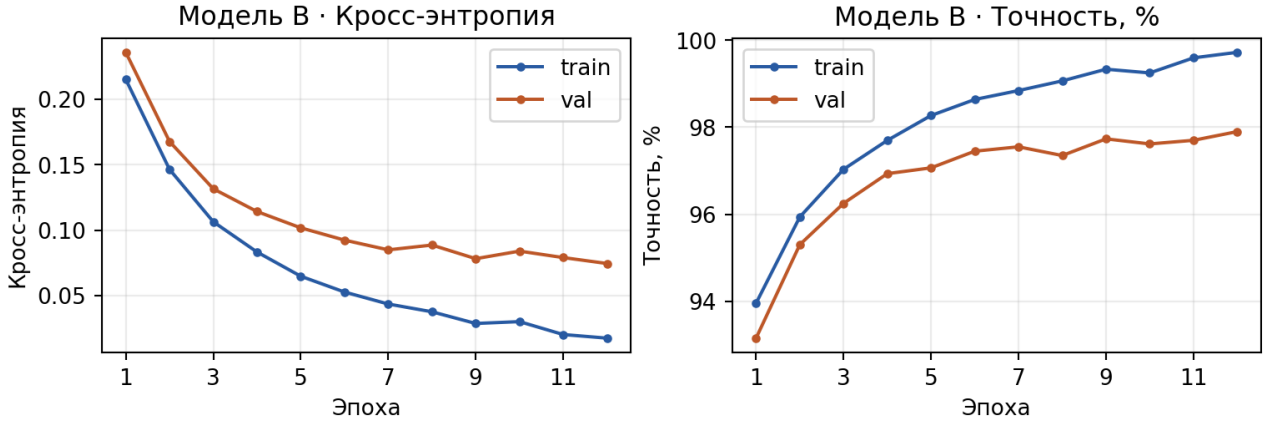


Рис. 3: Потери и точность модели B по эпохам

При увеличении ширины со 128 до 256 нейронов лучшая точность на валидации выросла на 0,52 п. п. К концу обучения модель B достигла меньшей кросс-энтропии и на train, и на validation.

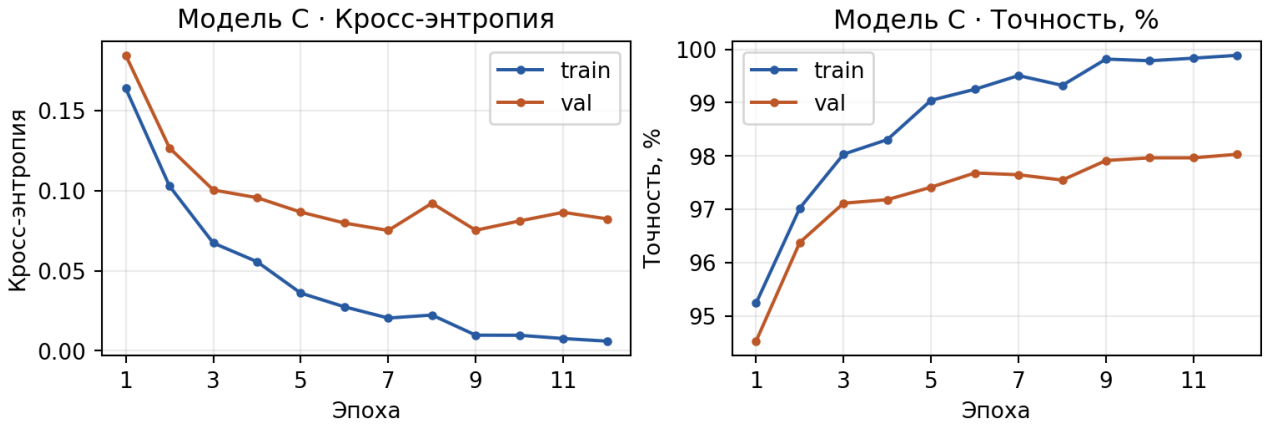


Рис. 4: Потери и точность модели C по эпохам

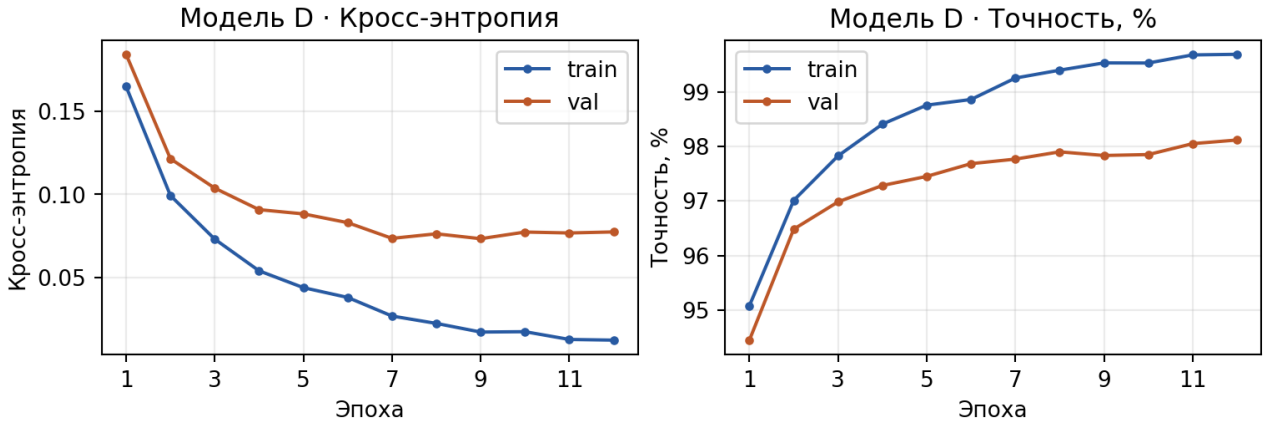


Рис. 5: Потери и точность модели D по эпохам

Второй скрытый слой дал ещё 0,13 п. п. У C с 9-й по 12-ю эпоху потери train продолжили снижаться, а потери val выросли с 0,07528 до 0,08240. Это совместимо с ростом уверенности в части ошибочных ответов.

При Dropout 0,2 разрыв train–val в конце обучения уменьшился с 1,85 до 1,57 п. п. Валидационная кросс-энтропия уменьшилась до 0,07744. Выигрыш D относительно C составил пять верных ответов на 6 000 изображений. Для тестовой оценки сохранены веса модели D с эпохи 12, на которой получена максимальная валидационная точность 98,12 %.

6. Тестовая оценка

Выбранная модель распознала 9 818 из 10 000 цифр. Точность составила 98,18 %, средняя кросс-энтропия — 0,06197. Строки матрицы ошибок соответствуют истинным меткам, столбцы — предсказанным.

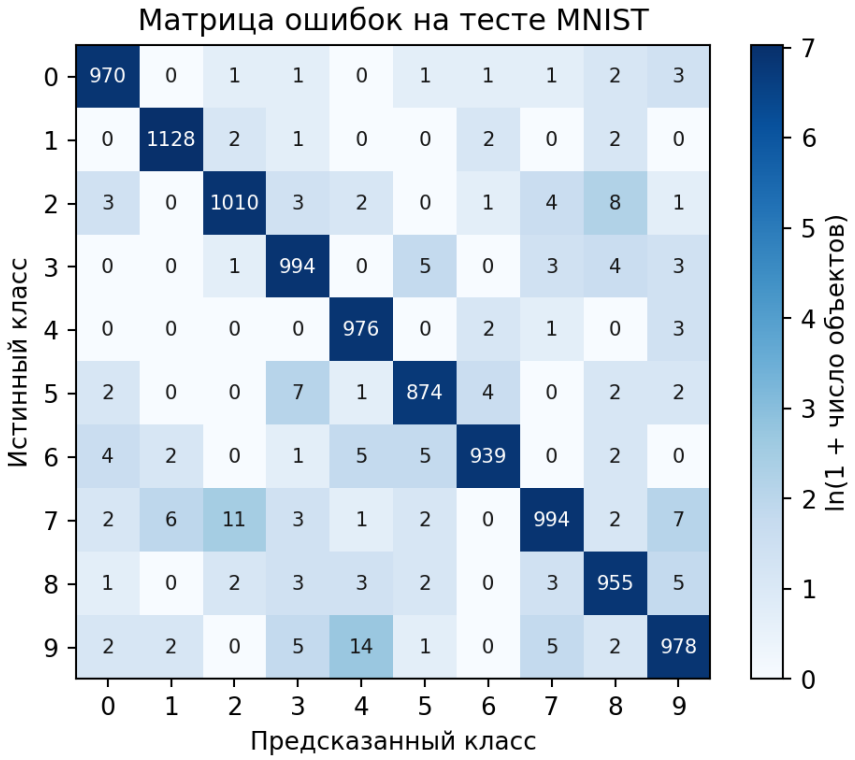


Рис. 6: Матрица ошибок на тестовой выборке

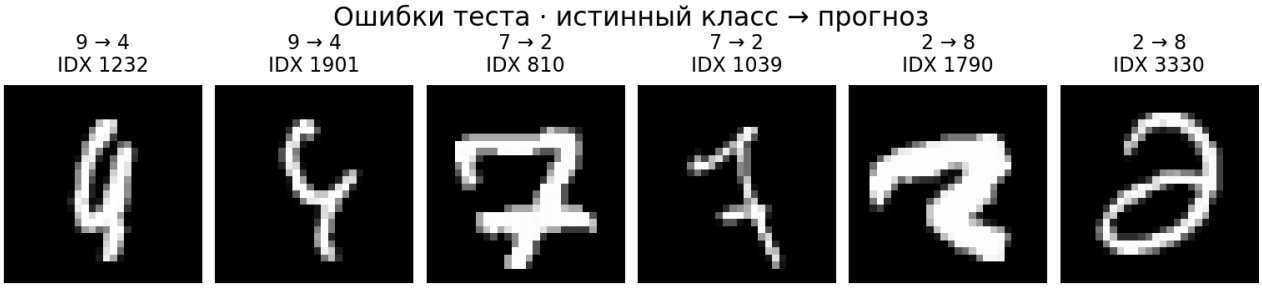


Рис. 7: Примеры ошибок из трёх наиболее частых направлений путаницы

Чаще всего модель принимала 9 за 4 (14 случаев), 7 за 2 (11) и 2 за 8 (8). У показанных 9 разомкнутый верх и нисходящий штрих напоминают открытую 4. Загиб верхнего штриха и поперечная черта у 7 могут восприниматься как верх и основание 2. У 2 с индексом 3330 нижний росчерк образует овал, а у 1790 толстые сближенные штрихи сужают просветы. Эти особенности могли сделать обе цифры похожими на 8.

7. Распознавание внешних изображений

Для проверки использованы три оригинальных цифровых рисунка, построенные по заданным вручную траекториям штрихов. У цифры 2 тонкий тёмный штрих и смещение влево, у 5 — светлый широкий штрих на тёмном фоне, у 7 — поперечная черта и смещение вниз и вправо.

Изображение переводится в оттенки серого. По медиане пикселей рамки оценивается фон, при необходимости изображение инвертируется. После вычитания фона область штриха вписывается в 20×20 с сохранением пропорций, переносится на холст 28×28 и сдвигается по центру масс к (13,5; 13,5). Интенсивность делится на 255. Softmax возвращает вероятности всех десяти классов.

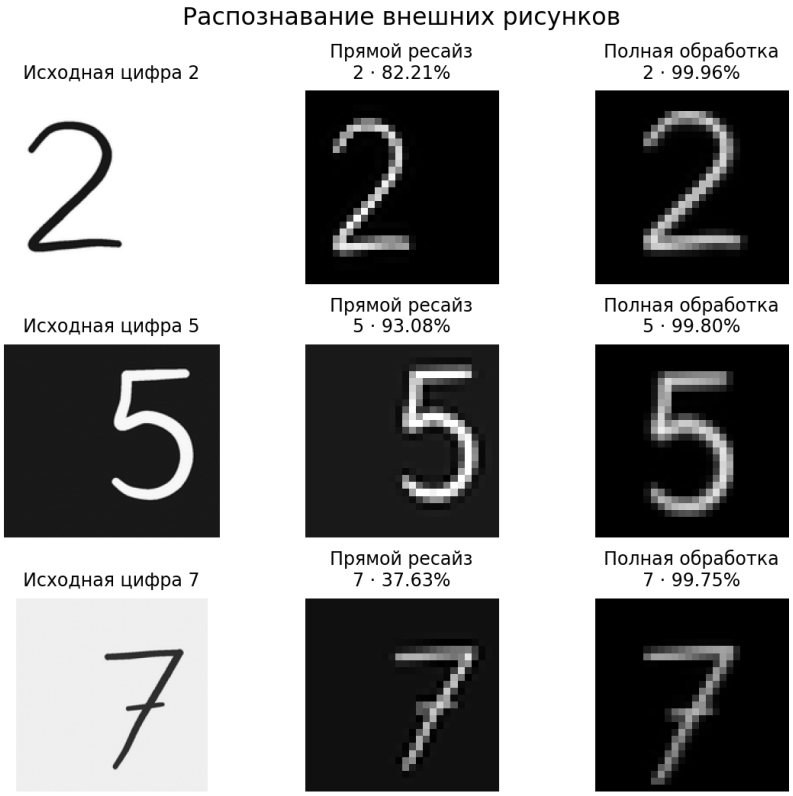


Рис. 8: Исходные рисунки, прямое масштабирование и полная обработка

Оба варианта обработки дали три верных ответа. Полная обработка повысила оценки Softmax для всех трёх рисунков. Для смещённой цифры 7 оценка выросла с 37,63 % до 99,75 %. Все преобразования проверялись вместе. На других рисунках низкий контраст, шум на границах или обрезанные штрихи могут помешать выделению цифры.

8. Заключение

В этом эксперименте расширение первого слоя дало больший прирост валидационной точности, чем добавление второго. Dropout 0,2 уменьшил разрыв между train и val, а модель с двумя скрытыми слоями и Dropout показала лучшую валидационную точность. На независимом тесте она распознала 98,18 % цифр. Сохранённые веса позволяют проверить новый рисунок и получить вероятности классов в ноутбуке без повторного обучения.